# Ledger Store

Technical Report — Features & Architecture | Vue 3 · TypeScript · Pinia · Vite · Tailwind CSS

---

## 1. Overview

Ledger Store is a single-page e-commerce storefront for a curated menswear catalog (shirts, shoes, watches), built with Vue 3 (Composition API, <script setup>), TypeScript, Pinia for state management, Vue Router for navigation, and Tailwind CSS for styling. Product and authentication data is sourced live from the public DummyJSON REST API. The application covers the full core shopping flow — browsing, filtering, searching, authentication, and cart management — with persisted state, toast notifications, dark mode, and a responsive layout including a dedicated mobile bottom tab bar.

## 2. Features Implemented

### Product Catalog & Discovery

- Live product catalog fetched from the DummyJSON API, curated to a men's shirts / shoes / watches selection.
- Full-text search across product title, description, and brand.
- Category filtering, including category-scoped routes (`/category/:category`).
- Price-range filtering and sorting by price, rating, and name.
- Product detail page with image gallery, stock/availability indicator, rating, reviews, and shipping/return policy information.

### Authentication

- Login flow against the DummyJSON auth API, with loading and error states on the login form.
- Session persistence via a stored bearer token, with session restoration on app load (`initAuth`).
- Route guarding: the cart route requires authentication and redirects unauthenticated users to `/login`, returning them afterward.

### Shopping Cart

- Per-user cart lines persisted to `localStorage`, reloaded automatically on login.
- Quantity management (add, increment/decrement, remove) with an “add to cart” action from both the product card and detail view.
- Cart state is cleared / reloaded appropriately around login and logout.

### Feedback, Theming & Responsiveness

- Toast notification queue for cart and authentication feedback (success / error / info), auto-dismissing.

- Dark mode with system-preference detection on first load, manual toggle, and persisted preference.
- Responsive layout with a dedicated bottom tab bar for mobile navigation, alongside the desktop nav bar.
- 404 fallback route (`NotFoundView`) for unmatched paths.

## Motion, Loading & Empty States

- Skeleton loading states for the product grid and product detail page while data is in flight.
- Dedicated empty-state and error-state components used across the catalog and cart.
- Scroll-reveal animations via an IntersectionObserver-based composable (`useScrollReveal`).
- Custom cursor hover interaction and a “magnetic” button effect for key call-to-action buttons.
- Animated page transitions between routes and a cart “bounce” animation on add-to-cart.

# 3. Architecture

The application follows a conventional Vue 3 SPA structure: a root shell (`App.vue`) hosts persistent chrome (nav, footer, toasts, mobile tab bar) around a router-driven content area. Route-level **views** compose smaller, reusable **components**, while cross-cutting state (catalog, cart, auth, theme, toasts) lives in focused **Pinia stores** rather than in the components themselves, keeping views declarative and stores independently testable.

## 3.1 Tech Stack

| Layer | Technology |
|---|---|
| Framework | Vue 3 (Composition API, <script setup> SFCs) |
| Language | TypeScript |
| Build tool | Vite |
| State management | Pinia (composition-style stores) |
| Routing | Vue Router (lazy-loaded routes, navigation guard) |
| Styling | Tailwind CSS |
| Data source | DummyJSON REST API (products + auth) |

## 3.2 Component Hierarchy

The diagram below shows how the root shell mounts persistent chrome components, how the router swaps in one of five route-level views, and which reusable components each view composes.

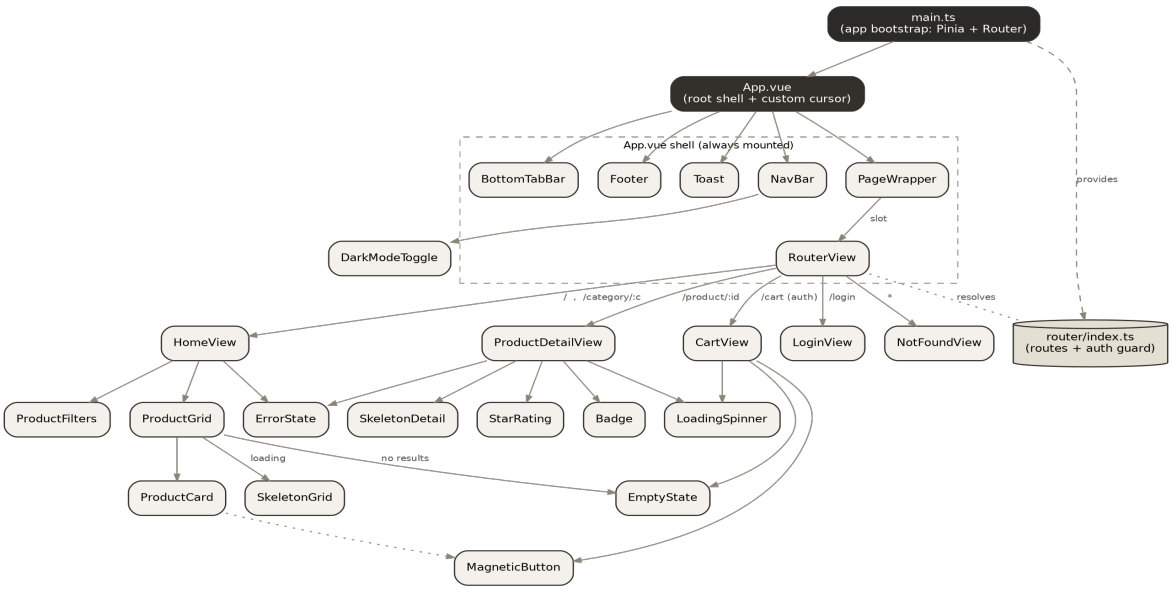


*Figure 1 — Component hierarchy: root shell, views, and their child components.*

## 3.3 State & Data Flow

State is organized into five focused Pinia stores. Views and components read from and write to these stores rather than talking to the API or `localStorage` directly, which keeps data-fetching and persistence logic in one place per concern.

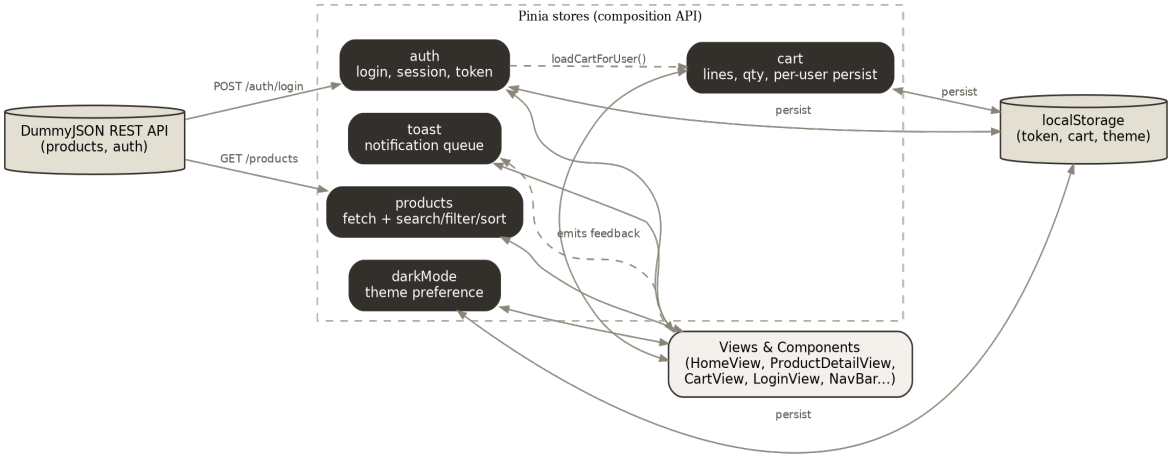


*Figure 2 — Store responsibilities and their connections to the API, localStorage, and the UI.*

## 3.4 Store Responsibilities

| Store | Responsibility |
|---|---|
| products | Fetches/caches the catalog; exposes filteredProducts (search + category + price + sort). |
| cart | Per-user cart lines, quantity logic, persisted to localStorage, reloaded on login. |
| auth | Login, session restoration from stored token, logout (incl. clearing cart state). |
| darkMode | Tracks and persists the active theme; respects system preference on first load. |
| toast | Simple queue for transient success / error / info notifications. |

## 3.5 Routing

| Path | View | Notes |
|---|---|---|
| `/` | `HomeView` | `Product catalog` |
| `/category/:category` | `HomeView` | `Filtered by category` |
| `/product/:id` | `ProductDetailView` | `Lazy-loaded` |
| `/cart` | `CartView` | `Requires authentication (route guard)` |
| `/login` | `LoginView` | `Redirects back after login` |

```
* (catch-all)          NotFoundView           404 fallback
```

### 3.6 Design System

- Typography: Fraunces (display/serif) paired with Inter (body) and IBM Plex Mono (accents).
- Color: neutral surface/edge/text scales with light and dark variants; muted beige for sale/discount indicators.
- Motion: custom fade / slide / scale keyframes, a cart bounce animation, and a shimmer effect for skeletons.